A Mini Project Report on

FraudGuard

Financial Fraud Detection & Customer Risk Analysis

Submitted in partial fulfilment of the requirements of the subject

Data Warehouse and Mining (DWM)

Submitted by

Omkar Padhi (Roll No: 24101C0003)

Shremaan Kumar (Roll No: 24101C0002)

Under the guidance of

Shruti Agrawal

Department of INFT

Vidyalankar Institute of Technology

Academic Year 2026–2027

ABSTRACT

Loan fraud causes direct financial loss to lenders and is difficult to detect because fraudulent applications form a very small share of all applications. FraudGuard is a web-based system that combines data warehousing concepts, data mining and machine learning to analyse loan applications and support the people who process them.

The system uses a historical dataset of 5,000 labelled loan applications with 25 attributes, of which 103 (2.06%) are labelled as fraud. The data is organised as a star schema with one fact table and five dimensions and is analysed through OLAP operations such as roll-up, drill-down, slice, dice and pivot. Five classifiers — Logistic Regression, Decision Tree, Gaussian Naive Bayes, Random Forest and XGBoost — are trained with a pipeline that applies preprocessing and SMOTE oversampling, and are evaluated on a held-out test set of 1,000 applications.

FraudGuard provides three role-based modules: a Customer module to apply for a loan and track its status, a Bank Officer module to review applications and approve, reject or send them for review with a complete audit trail, and an Admin / Technical module that exposes the dataset profile, data quality checks, model comparison and OLAP analysis.

The evaluation shows that the dataset is highly imbalanced and that the available features have a very weak relationship with the fraud label: the highest ROC-AUC obtained is 0.5553 (XGBoost) and the highest recall is 38.1% (Logistic Regression). The report presents these results as measured, discusses their limitations and outlines future work.

Keywords: fraud detection, data warehouse, star schema, OLAP, data mining, classification, class imbalance, SMOTE, XGBoost, role-based access control.

1. Introduction and Motivation

Banks and non-banking finance companies receive a large number of loan applications every day. A small fraction of these are fraudulent: the applicant misrepresents income, uses a synthetic identity, takes several loans at the same time, or uses the loan to move illegitimate money. Every such application that is approved becomes a direct loss, while every genuine application that is wrongly rejected is a lost customer.

Data Warehouse and Mining (DWM) provides the concepts needed to study this problem systematically. A data warehouse organises historical application data around the business process being analysed, On-Line Analytical Processing (OLAP) lets an analyst view the same facts from several dimensions, and classification attempts to learn the difference between fraudulent and genuine applications from labelled history. FraudGuard applies these concepts in one working system and reports its results as measured, including the cases where the models perform poorly, because understanding why a model does not work is as much a part of data mining as building it.

2. Problem Statement and Objectives

In the conventional process a loan officer checks an application using personal judgement and a few fixed rules. Such rules do not adapt to new fraud patterns, similar applications can receive different outcomes, the reason for a decision is often not recorded, the applicant cannot see the status of the application, and historical data kept as flat records cannot easily answer questions such as which loan type had the highest fraud rate in a given quarter. Fraudulent applications are rare, so they are easily missed.

Problem statement: *to design and implement a system that (a) organises historical loan-application data for multidimensional analysis, (b) applies and honestly evaluates classification models for detecting fraudulent applications on highly imbalanced data, and (c) supports the complete loan-application workflow for customers and bank officers with a traceable record of every decision.*

The objectives of the project are:

1. To describe a real loan-application dataset as a data warehouse star schema with fact and dimension tables.
2. To perform exploratory data analysis and OLAP operations (roll-up, drill-down, slice, dice and pivot) on fraud across time, loan type, employment, gender and age group.
3. To build a reproducible machine-learning pipeline with preprocessing, handling of class imbalance and five classification algorithms.
4. To evaluate and compare the models using accuracy, precision, recall, F1 score, ROC-AUC, PR-AUC and confusion matrices on a held-out test set.
5. To implement Customer, Bank Officer and Admin / Technical modules protected by role-based access control, with an audit trail of every decision.

3. Proposed System and Architecture

FraudGuard replaces the manual process with a single web application that has three role-based modules sharing one backend. Every application receives a fraud probability from a saved machine-learning pipeline, which is converted into a risk level (Low below 25%, Medium from 25% to below 50%, High from 50%). Decisions follow the states New, Under Review, Approved and Rejected, and every officer action is stored with its time, reason and decision maker. All figures in the interface are computed from the data on request; where a value cannot be computed, the interface shows “Not available”.

Table 1: Modules of the proposed system

Module	User	Purpose
Customer	Loan applicant	Sign up, apply for a loan through a five-step form, track status and timeline, and see the reason when an application is rejected.
Bank Officer	Loan officer	Work through the queue of new applications, see the fraud probability and risk level, and approve, reject (with a reason) or send for review.
Admin / Technical	Administrator, ML engineer, evaluator	Read-only technical view of the dataset, data quality, ML model comparison, feature analysis, OLAP analysis and system information.

The system follows a three-tier architecture (Figure 1): a browser-based presentation tier built with React and TypeScript, an application tier that exposes a FastAPI REST API, and a data tier consisting of the historical dataset, a SQLite operational database and the saved model files. After sign-in the backend issues a session token that the browser sends with every request. Each of the 21 API endpoints declares which role may call it, and the backend rejects any other role. The administrator signs in on a separate admin sign-in page.

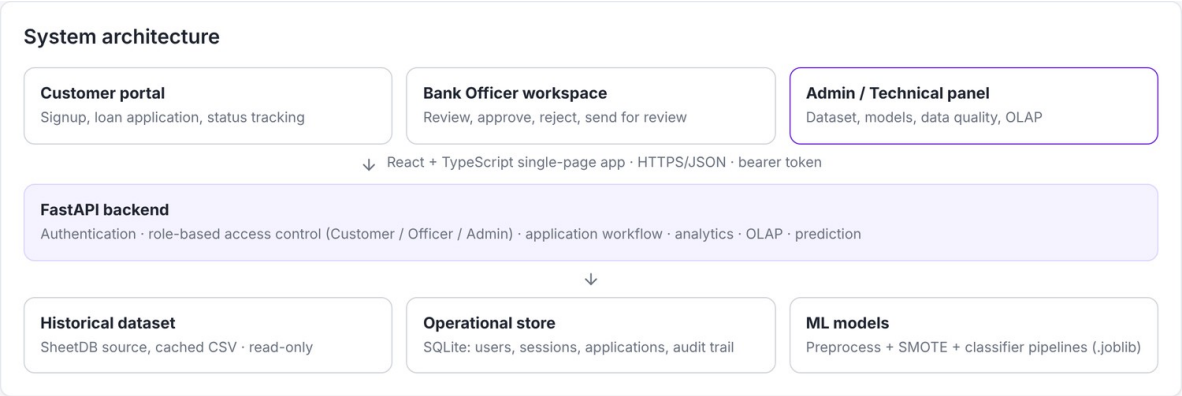


Figure 1: System Architecture

Figure 2 shows how data moves through the system. The historical dataset is read-only analytical data: it is loaded from the configured SheetDB source (or from the cached copy data/cleaned_dataset.csv if the source cannot be reached) and aggregated in memory with pandas. Operational data created through the application is stored in four SQLite tables: users, sessions, applications and application_events. A new application is validated, scored by the selected model, placed in the officer's queue, and every later decision is added to the audit trail in the same transaction as the status change.

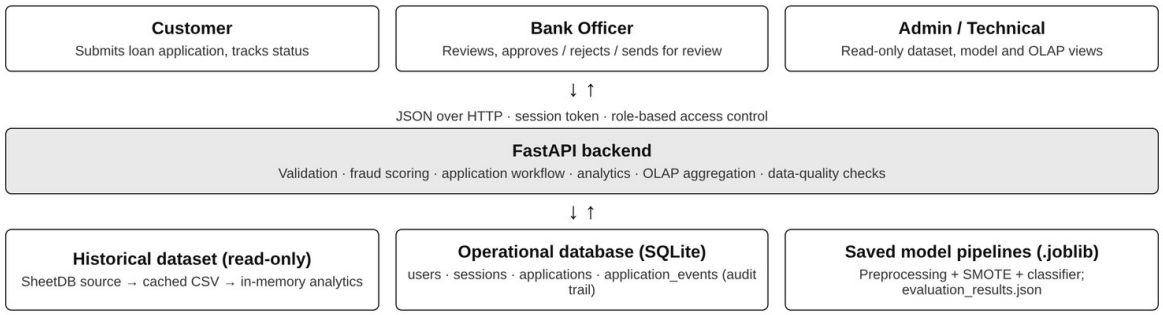


Figure 2: Database and Data Flow

4. Dataset and Data Preprocessing

The historical dataset contains 5,000 loan applications and 25 columns, covering applications dated 5 June 2022 to 4 June 2025. Each record represents one application; there are 5,000 distinct customer identifiers. The target column is FraudFlag (1 = fraud, 0 = genuine). The dataset does not contain a CIBIL (credit) score or the applicant's name, so no model uses a credit score and historical records are identified by Customer ID.

Table 2: Summary of the dataset

Property	Value	Property	Value
Records	5,000	Columns	25 (12 numerical, 13 categorical / text)
Fraud records	103 (2.06%)	Genuine records	4,897 (97.94%)
Missing values	0	Duplicate rows	0
Date range	5 June 2022 to 4 June 2025	Average loan amount	₹5,10,508

Table 3: Attribute groups of the dataset

Group	Columns
Identifiers	ApplicationID, CustomerID
Applicant	ApplicantAge, Gender, NumberDependents, EmploymentStatus, PropertyOwnershipStatus, ResidentialAddress
Loan	LoanType, PurposeOfLoan, LoanAmountRequested, LoanTenureMonths, InterestRateOffered
Financial	MonthlyIncome, ExistingEMIs, DebtToIncomeRatio
Time	ApplicationDate, DateKey, Day, Month, Quarter, Year
Outcome	FraudFlag (target), FraudType, LoanStatus

Preprocessing is implemented in the training script (ml/train.py) and is saved together with each model as a single pipeline, so that exactly the same steps are applied when a new application is scored.

Table 4: Preprocessing steps

Step	Description
Cleaning	Exact duplicate rows are dropped; numerical columns and FraudFlag are converted to numbers; a missing DebtToIncomeRatio is computed as ExistingEMIs ÷ MonthlyIncome × 100.
Feature selection	13 input features: 8 numerical (ApplicantAge, NumberDependents, LoanAmountRequested, LoanTenureMonths, InterestRateOffered, MonthlyIncome, ExistingEMIs, DebtToIncomeRatio) and 5 categorical (Gender, EmploymentStatus, LoanType, PurposeOfLoan, PropertyOwnershipStatus). Identifiers, dates, address, FraudType and LoanStatus are excluded; the last two would leak the label.
Transformation	Median / most-frequent imputation, standardisation of numerical features and one-hot encoding of categorical features, giving 32 encoded features.
Train-test split	Stratified 80 / 20 split with a fixed random seed: 4,000 training and 1,000 test applications.
Class imbalance	SMOTE oversampling is applied to the training fold only, inside the pipeline, so that no synthetic record enters the test set. Class weights are also used where the algorithm supports them.

Data quality. The Admin / Technical module runs 8 checks on the loaded dataset and assigns each a status from a stated rule: 5 are Healthy (missing values, duplicates, invalid categories, data types, cross-column consistency), 1 is a Warning (one DebtToIncomeRatio of 102% is out of range) and 2 Need Attention (class imbalance, because fraud is only 2.06% of the records, and statistical outliers, because 405 debt-to-income values lie outside the 1.5 × IQR band; these are legitimate extreme values and are not removed).

5. Exploratory Data Analysis

Exploratory analysis was carried out on all 5,000 records. The fraud percentage of a group is the number of fraud records in the group divided by the number of records in the group. Figure 3 shows the fraud percentage for each quarter and Table 5 the breakdown by the main categorical dimensions.

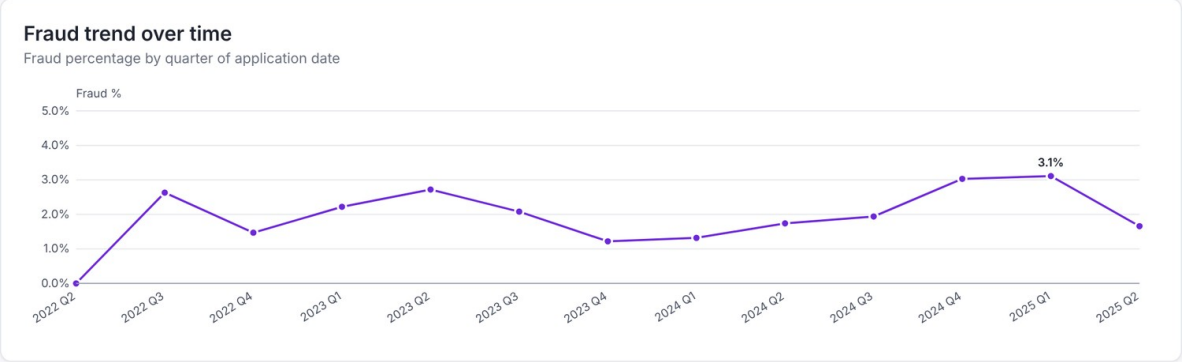


Figure 3: Fraud Percentage by Quarter of Application Date

Table 5: Fraud by loan type, employment status, gender and age group

Dimension	Group	Applications	Fraud cases	Fraud %
Loan type	Business Loan	957	22	2.30
	Education Loan	1,027	22	2.14
	Personal Loan	1,026	22	2.14
	Home Loan	1,033	20	1.94
	Car Loan	957	17	1.78
Employment	Salaried	835	22	2.63
	Self-Employed	808	21	2.60
	Student	831	19	2.29
	Business Owner	816	16	1.96
	Unemployed	833	14	1.68
	Retired	877	11	1.25
Gender	Female	1,679	42	2.50
	Other	1,710	33	1.93
	Male	1,611	28	1.74
Age group	18-25	565	13	2.30
	26-35	1,082	17	1.57
	36-45	1,098	26	2.37
	46-55	1,089	30	2.75
	56+	1,166	17	1.46

Observations:

- Fraud percentage varies only slightly between groups: from 1.78% (Car Loan) to 2.30% (Business Loan) across loan types, and from 1.25% (Retired) to 2.63% (Salaried) across employment categories. Each group contains only between 11 and 42 fraud cases, so these differences should not be over-interpreted.
- The Pearson correlation of every numerical feature with FraudFlag is close to zero; the largest in absolute value is -0.0166 (LoanTenureMonths).
- The quarterly fraud percentage ranges from 0.00% (2022 Q2, which contains only 104 applications) to 3.11% (2025 Q1) without a clear trend.
- The four recorded fraud types are Income Misrepresentation (30), Loan Stacking (26), Synthetic Identity (25), Transaction Laundering (22).

6. Fraud Detection Methodology and ML Models

Fraud detection is treated as a supervised binary classification problem: given the features of an application, predict whether FraudFlag is 1 or 0. Five algorithms are trained with the same preprocessing, the same training data and the same test data so that their results can be compared directly (Figure 4).

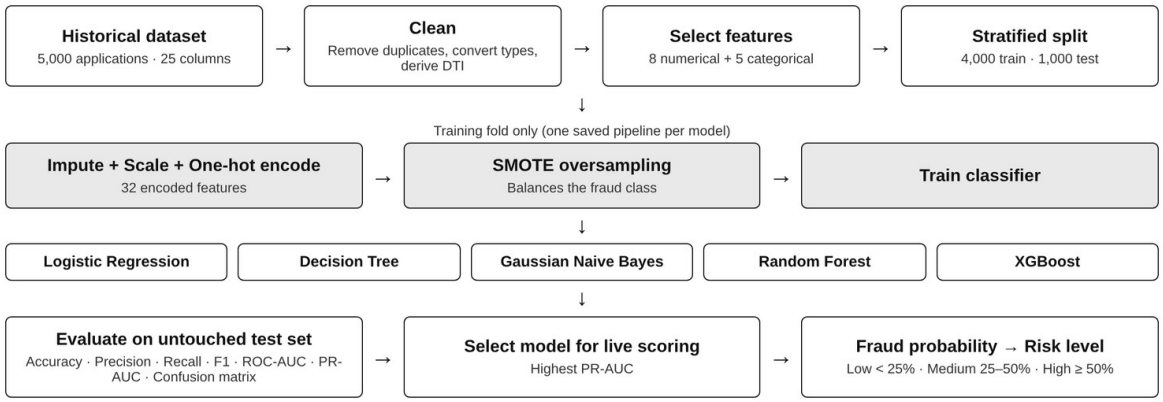


Figure 4: ML Workflow

Table 6: Models and their configuration

Model	Family	Main settings
Logistic Regression	Linear model	max_iter = 1500, class_weight = balanced
Decision Tree	Tree-based model	max_depth = 8, class_weight = balanced, criterion = gini
Gaussian Naive Bayes	Probabilistic model	Default settings
Random Forest	Tree ensemble (bagging)	n_estimators = 250, class_weight = balanced
XGBoost	Tree ensemble (gradient boosting)	n_estimators = 250, max_depth = 5, learning_rate = 0.05, subsample = 0.85, colsample_bytree = 0.85, scale_pos_weight ≈ 47.78

When a customer submits an application, the backend validates every field, derives the debt-to-income ratio and passes the application to the saved pipeline of the selected model, which returns a fraud probability. The probability is classified as Fraud when it is 0.5 or more and is mapped to a risk level. The bank officer sees the probability, the risk level and the application details and takes the decision; the model supports the officer and never approves or rejects an application by itself. The model used for live scoring is the one the training script marks as selected (highest PR-AUC), which in the current results is XGBoost. Saved model files are present for Logistic Regression, Decision Tree, Gaussian Naive Bayes and XGBoost; the Random Forest file is not part of the project folder, so only its recorded evaluation results are reported.

Global and individual explanations. The system shows global feature importance, which describes what a model relies on overall. It does not explain why one particular applicant received a particular score, and the interface states this explicitly.

7. Model Evaluation

All models were evaluated on the same held-out test set of 1,000 applications, which contains 21 fraud cases and 979 genuine cases. A probability of 0.5 or more is counted as a fraud prediction. The values are taken from models/evaluation_results.json, the file written by the training script.

Table 7: Evaluation metrics of the five models on the test set

Model	Accuracy	Precision	Recall	F1 score	ROC-AUC	PR-AUC
Logistic Regression	60.6%	2.1%	38.1%	3.9%	0.5072	0.0221
Decision Tree	85.1%	0.8%	4.8%	1.3%	0.4412	0.0181
Gaussian Naive Bayes	66.4%	2.1%	33.3%	4.0%	0.5189	0.0228
Random Forest	97.9%	0.0%	0.0%	0.0%	0.5353	0.0248
XGBoost	94.7%	0.0%	0.0%	0.0%	0.5553	0.0288

Table 8: Confusion matrices on the test set

Model	True negative	False positive	False negative	True positive
Logistic Regression	598	381	13	8
Decision Tree	850	129	20	1
Gaussian Naive Bayes	657	322	14	7
Random Forest	979	0	21	0
XGBoost	947	32	21	0

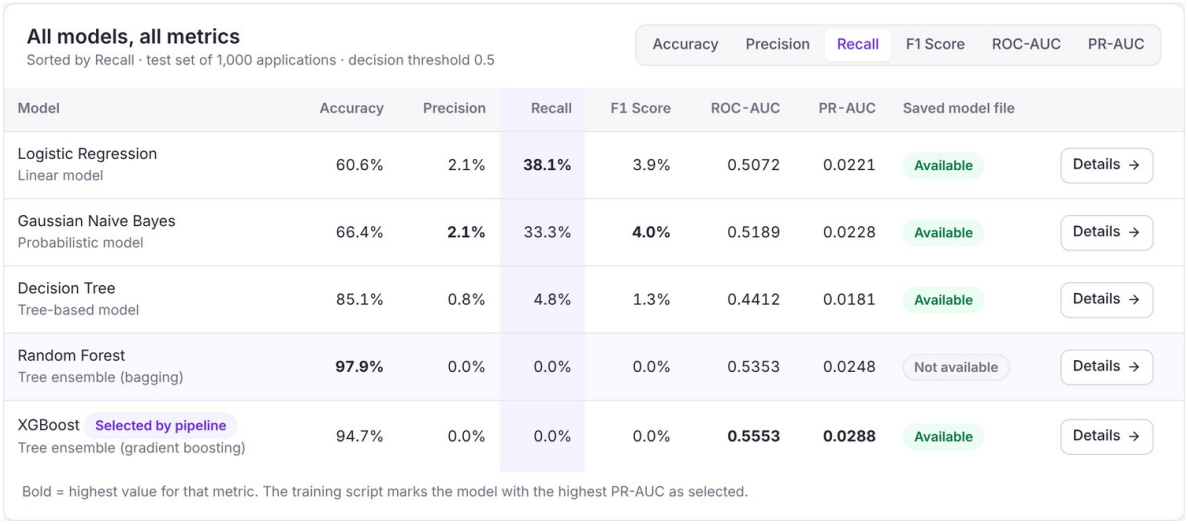


Figure 5: ML Model Comparison

- Random Forest has the highest accuracy (97.9%) but detects none of the 21 fraud cases; it predicts “genuine” for every test application. XGBoost (94.7% accuracy) also detects no fraud case and raises 32 false alarms. Accuracy is therefore misleading on this dataset.
- Logistic Regression detects the most fraud cases (8 of 21, recall 38.1%), but at the cost of 381 false positives, giving a precision of only 2.1%. Gaussian Naive Bayes has the highest precision (2.1%) and F1 score (4.0%).
- The ROC-AUC values lie between 0.4412 and 0.5553; 0.5 corresponds to random ranking. The PR-AUC values (0.0181 to 0.0288) are close to the fraud rate of the test set (0.021), the value a random classifier would obtain.
- With only 21 fraud cases in the test set, one additional detected case changes recall by 4.8 percentage points, so all metrics have high variance. For these reasons no model is described as the “best” model; the comparison page lets the user sort the models by any metric.
- Global feature importance is available for the tree-based models and, as the absolute coefficient, for Logistic Regression; Gaussian Naive Bayes does not provide it. The

importances are spread thinly over many encoded features and differ from model to model, which is consistent with the weak relationship between the features and the label.

8. Fraud Analytics and DWM / OLAP Analysis

The Fraud Analytics page, available to the bank officer and the administrator, is computed from the historical dataset each time it is opened. It shows the fraud percentage (2.06%), fraud cases (103), genuine cases (4,897) and average loan amount (₹5,10,508), the fraud cases of each recorded type, the fraud percentage by quarter, loan type, employment status, gender and age group, and the evaluation metrics of all models. The officer also has a Fraud Checker page on which a single hypothetical application can be scored; nothing is stored by that page.

For warehouse-style analysis the data is modelled as a star schema whose grain is one row per loan application (Figure 6). The fact table FACT_LOAN_APPLICATION holds the measures LoanAmountRequested, MonthlyIncome, ExistingEMIs, DebtToIncomeRatio and FraudFlag, and is surrounded by five dimensions: DIM_DATE (Day, Month, Quarter, Year), DIM_CUSTOMER (age group, gender, dependents, property ownership), DIM_LOAN (loan type, purpose, tenure, interest rate), DIM_EMPLOYMENT (employment status) and DIM_FRAUD_STATUS (fraud flag, fraud type, loan status). The star schema is a documented analytical design: the tables are not materialised in a separate warehouse database, and the equivalent aggregations are computed by the backend over the dataset.

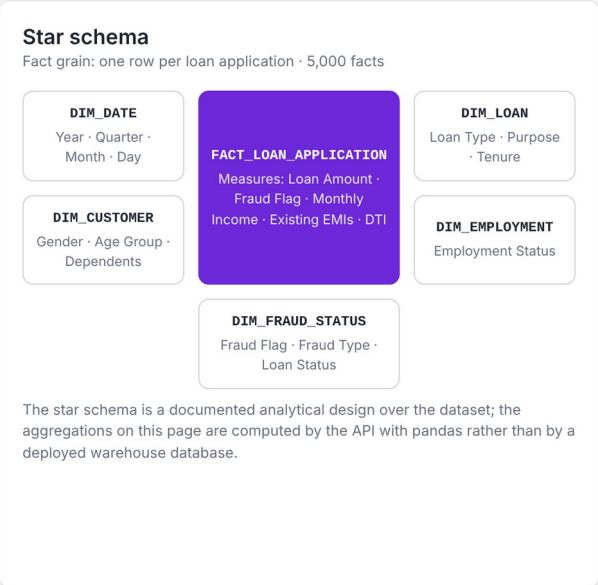


Figure 6: Star Schema used for OLAP Analysis

Table 9: OLAP operations implemented

Operation	Implementation in FraudGuard
Roll-up	Month to Quarter to Year; individual ages are rolled up to five age groups.
Drill-down	Year → Quarter → Month → Loan Type, with the fraud cases shown at every level.
Slice	One dimension is selected and the measures are aggregated over all other dimensions.
Dice and pivot	Filters on several dimensions are combined, and any two dimensions can be placed on the rows and columns of a cross-tabulation for a chosen measure.

Table 10: Roll-up to the Year level

Year	Loan count	Fraud count	Fraud %	Total loan amount	Average loan amount
2022	932	17	1.82	₹47,48,58,000	₹5,09,504
2023	1,652	34	2.06	₹83,64,04,000	₹5,06,298
2024	1,664	33	1.98	₹85,29,69,000	₹5,12,602
2025	752	19	2.53	₹38,83,09,000	₹5,16,368
Total	5,000	103	2.06	₹2,55,25,40,000	₹5,10,508

Table 11: Pivot: fraud percentage by loan type and year

Loan type	2022	2023	2024	2025	All years
Home Loan	0.00	2.08	1.76	4.12	1.94
Education Loan	3.35	1.73	2.20	1.44	2.14
Personal Loan	1.53	2.47	1.74	3.09	2.14
Business Loan	2.60	2.27	2.51	1.46	2.30
Car Loan	1.68	1.79	1.68	2.08	1.78
All loan types	1.82	2.06	1.98	2.53	2.06

9. Application Modules

9.1 Customer Module

A customer creates an account with name, email and password (stored only as a salted hash) and receives a Customer ID. The Apply for Loan page is a five-step form — Personal Information, Employment, Financial Information, Loan Information, and Review & Submit — with validation of each field, automatic calculation of the debt-to-income ratio, a warning before leaving an unfinished form and protection against duplicate submission. My Applications lists each application with its ID, date, loan type, amount, status and timeline; when an application is rejected, the reason and the officer’s note are shown. The customer never sees the fraud probability, the risk level or internal review notes, and cannot open another customer’s application.

9.2 Bank Officer Module

The bank-officer navigation contains Dashboard, New Applications, All Applications, Fraud Analytics, Fraud Checker, Data Explorer and Settings. The dashboard shows live counts of total, new, under-review, approved and rejected applications (Figure 7). New Applications is the work queue, with its count shown as a badge that changes as decisions are taken. All Applications combines historical and portal applications with status tabs, search, filters, sorting and pagination; historical records are read-only. The review screen shows the applicant, employment, financial and loan information, the fraud analysis, the timeline and the decision history.

Table 12: Application status transitions

From	Action	To	Condition
New	Send for Review	Under Review	Reason required
New / Under Review	Approve	Approved	Confirmation
New / Under Review	Reject	Rejected	Reason from a fixed list required

Each action is stored in the audit trail with the previous and new status, the reason, the note, the time and the decision maker, and is never overwritten. A decision on an application that has already been decided by someone else is refused.

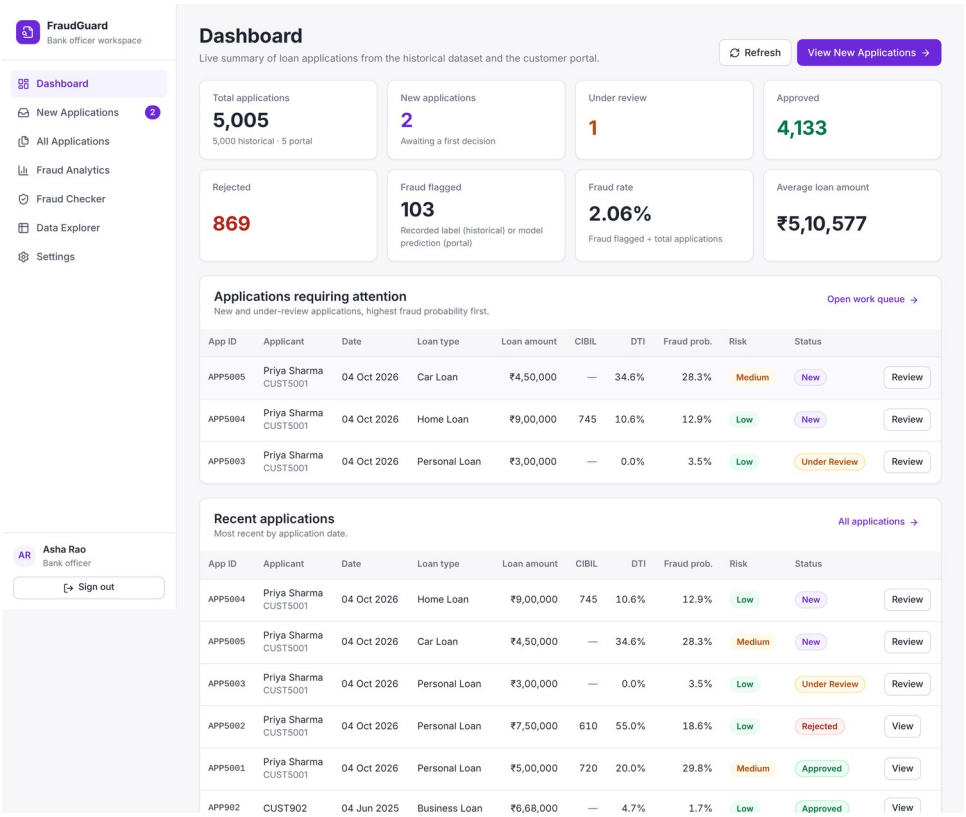


Figure 7: Bank Officer Module — Dashboard

9.3 Admin / Technical Module

The admin / technical module is intended for the project administrator, an ML engineer or an evaluator. It is separate from the bank-officer module: it has its own role, its own sign-in page and its own API endpoints, and it is read-only — an administrator cannot approve or reject applications. Its eleven pages are Admin Dashboard (Figure 8), Dataset Overview, Data Explorer, Fraud Analytics, ML Model Comparison, Model Performance (confusion matrix, ROC and precision-recall curves, hyper-parameters, feature importance and limitations of one model), Feature Analysis, Data Quality, DWM / OLAP Analysis, System / Technical Info and Settings.

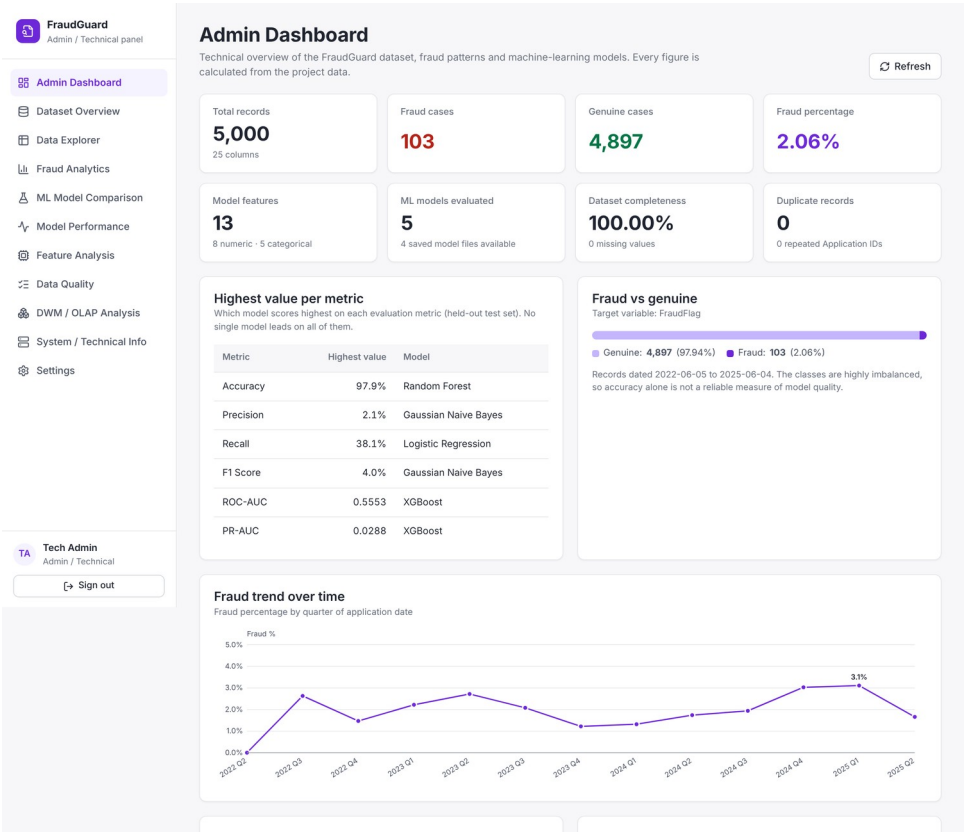


Figure 8: Admin / Technical Module — Dashboard

Table 13: Role-based access control

Area	Customer	Bank Officer	Admin
Own applications (apply, track)	Yes	No	No
Application queue, review, decisions and Fraud Checker	No	Yes	No
Fraud analytics, data explorer, model metrics	No	Yes	Yes
Dataset profile, data quality, OLAP, feature analysis, system information	No	No	Yes

10. Results and Limitations

10.1 Results

- A working web application with three role-based modules, a REST backend with 21 endpoints and a persistent operational database was implemented. The complete workflow New → Under Review → Approved or Rejected was tested, including the mandatory rejection reason, the audit trail and the customer's view of the outcome; automated end-to-end tests of all three roles, including access-control checks, pass on the delivered version.
- The dataset of 5,000 applications is complete (0 missing values, 0 duplicate rows) and contains 103 fraud cases (2.06%). Fraud percentage differs only slightly between loan types, employment categories, genders and age groups, and the OLAP page reproduces these aggregations at the Year, Quarter, Month and Loan Type levels.
- The main modelling result is negative: with the features available in this dataset, none of the five models detects fraud reliably. The two models with the highest accuracy detect no fraud at all, and the model with the highest recall (38.1%) produces 381 false alarms. This

demonstrates why accuracy must not be used alone on imbalanced data and why recall, precision and the area under the curves have to be examined together.

Table 14: Summary of model results

Model	Recall	Precision	ROC-AUC	Fraud cases detected
Logistic Regression	38.1%	2.1%	0.5072	8 of 21
Decision Tree	4.8%	0.8%	0.4412	1 of 21
Gaussian Naive Bayes	33.3%	2.1%	0.5189	7 of 21
Random Forest	0.0%	0.0%	0.5353	0 of 21
XGBoost	0.0%	0.0%	0.5553	0 of 21

10.2 Limitations

- Predictive performance is low: the ROC-AUC of every model is close to 0.5, so the fraud probability shown to the officer should be treated as a weak indicator only.
- The dataset contains only 103 fraud cases, of which 21 are in the test set, and has no credit score, repayment history or document-verification attributes.
- The model selected for live scoring has a recall of 0% at the 0.5 threshold; the decision threshold has not been tuned. The saved Random Forest model file is not present.
- Only global feature importance is available; explanations for individual applications are not implemented.
- The star schema is not materialised in a warehouse database; aggregations are computed in memory over a single dataset.
- The CIBIL score entered by a portal applicant is self-declared, unverified and not used by the models. The system is a prototype using SQLite on a single machine and has not been tested for load or audited for security.

11. Future Scope and Conclusion

11.1 Future Scope

- Obtain richer features (credit history, repayment behaviour, device and document verification signals) and re-evaluate the models.
- Tune the decision threshold for a target recall and use cross-validation to obtain more stable estimates.
- Add explanations for individual predictions, for example with SHAP values.
- Materialise the star schema in a relational warehouse (for example PostgreSQL) with scheduled extract-transform-load jobs.
- Apply other data-mining techniques such as association-rule mining and clustering for anomaly detection.
- Add document upload and verification, customer notifications and periodic retraining with performance monitoring, and deploy with a production database and HTTPS.

11.2 Conclusion

FraudGuard brings together the main topics of the Data Warehouse and Mining course in one working system. The loan-application data was modelled as a star schema and analysed with OLAP operations; data-quality checks and exploratory analysis were carried out on 5,000 records; five

classification models were trained with a reproducible pipeline and compared on six metrics; and the results are presented through a Customer module, a Bank Officer module and an Admin / Technical module protected by role-based access control.

The warehouse and workflow parts of the project meet their objectives. The modelling part shows that, for this dataset, the available attributes do not allow fraud to be predicted reliably, and that a high accuracy can hide a model that detects no fraud. Reporting this result as it is, and identifying the data that would be needed to improve it, is the main learning outcome of the project.

References

[1] J. Han, M. Kamber and J. Pei, Data Mining: Concepts and Techniques, 3rd ed. Morgan Kaufmann, 2011.

[2] R. Kimball and M. Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd ed. Wiley, 2013.

[3] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[4] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[5] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] G. Lemaître, F. Nogueira and C. K. Aridas, “Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning,” Journal of Machine Learning Research, vol. 18, no. 17, pp. 1–5, 2017.

[8] J. Davis and M. Goadrich, “The Relationship Between Precision-Recall and ROC Curves,” in Proc. 23rd Int. Conf. on Machine Learning, 2006, pp. 233–240.

[9] R. J. Bolton and D. J. Hand, “Statistical Fraud Detection: A Review,” Statistical Science, vol. 17, no. 3, pp. 235–255, 2002.

[10] FastAPI documentation, https://fastapi.tiangolo.com; React documentation, https://react.dev; pandas documentation, https://pandas.pydata.org/docs.